

we think about the OSI layer model, we think about the transport layer protocols, which are TCP, UDP and SSL.

Protocol: In addition to transport, we have a higher level of the protocol too. Once you have received some bytes from the network, protocols describe how to process network events asynchronously. These map to the OSI application layer protocol. Examples are HTTP, IMAP, IRC and DNS.

Twisted is written in Python 2.7; however, some parts of it do not get converted to Python 3.7. In this article, I am going to demonstrate the DNS server, which gives a dynamic response. We are going to use Python 3 to run the code. First, install Twisted in Python by using the following command:

```
Pip install twisted
```

If Pip works for Python 2, then you can use the following command:

```
python3 -m pip install twisted
```

After installing Twisted in Python 3, run the following code, which is also available at <https://github.com/mohitraj/mohitcs/blob/master/dns1.py>.

Import the following modules:

```
from twisted.internet import reactor,
defer
from twisted.names import client, dns,
error, server
import argparse
```

The following class calculates the answers to certain queries based on the query type and name:

```
class DNS_Reslover(object):
    def __init__(self, p,ip):
        self._pattern = p
        self._network = ip
    def _dynamicResponseRequired(self,
query):
```

Check the query to determine if a

dynamic response is required.

```
        if query.type == dns.A:
            labels = query.name.name.
            split(b'.'.)
            #import pdb;pdb.set_trace()
            if labels[0].
startswith(self._pattern.encode()):
                return True
            return False
```

The following method calculates the response to the query:

```
    def _doDynamicResponse(self,
query):
        name = query.name.name
        labels = name.split(b'.'.)
        parts = labels[0].split(self._
pattern.encode())
        lastOctet = parts[1]
        #import pdb;pdb.set_trace()
        address=self._network.
encode()+b"."+ lastOctet

        answer = dns.RRHeader(
            name=name,
            payload=dns.
Record_A(address))

        answers = [answer]
        authority = []
        additional = []
        return answers, authority,
additional
```

The query method checks if the query should be answered dynamically; otherwise, it is dispatched to the fallback resolver.

```
    def query(self, query,
timeout=None):
        if self._
dynamicResponseRequired(query):
            return defer.succeed(self._
doDynamicResponse(query))
        else:
            return defer.fail(error.
DomainError())
```

The *main()* function runs the server:

```
def main():
    factory = server.DNSServerFactory(
        clients=[DNS_
Reslover(pattern,IP), client.
Resolver(resolv='/etc/resolv.conf')]
    )
    protocol = dns.DNSDatagramProtocol(
controller=factory)
    reactor.listenUDP(53, protocol)
    reactor.listenTCP(53, factory)
    reactor.run()
```

The *main_arg()* function creates the command line arguments:

```
def main_arg():
    version1 = 1.0
    parser = argparse.ArgumentParser()
    parser.add_argument('-o', nargs=2,
help='First give pattern then IP')
    parser.add_argument('-v',
help="Version Number",action='store_
const',const=version1 )

    args = parser.parse_args()
    try :
        if args.o and len(args.o)==2:
            pattern, IP= args.o
            main(pattern,IP)

        elif args.v:
            print ("Current Version is
",args.v)
        else :
            print ("Use -h for the
options")
            print ("For more help email to
mohitraj.cs@gmail.com")
    except Exception as e :
        print (e)

if __name__ == '__main__':
    main_arg()
```

You can run the code as shown in the following command:

```
Python dns1.py -o workstation123.
mohit.com 172.16.2
```

Let's convert the above code to